HOME WORK SET 5

C Programming / Data Structures

Roll Number: 2025115140

Programs with source code and executed terminal-output screenshots

1. Polynomial Addition using Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int coeff, exp; struct Node *next; };

void insert(struct Node **h,int c,int e) {
    struct Node *n=malloc(sizeof(struct Node)),*p=*h;
    n->coeff=c; n->exp=e; n->next=NULL;
    if(!*h){*h=n;return;}
    while(p->next)p=p->next;
    p->next=n;
}

struct Node *add(struct Node *p,struct Node *q) {
    struct Node *r=NULL;
    while(p&&q) {
        if(p->exp==q->exp) {
            insert(&r,p->coeff+q->coeff,p->exp);
            p=p->next;q=q->next;
        } else if(p->exp>q->exp) {
            insert(&r,p->coeff,p->exp); p=p->next;
        } else {
            insert(&r,q->coeff,q->exp); q=q->next;
        }
    }
    while(p){insert(&r,p->coeff,p->exp);p=p->next;}
    while(q){insert(&r,q->coeff,q->exp);q=q->next;}
    return r;
}

void display(struct Node *p){
    while(p){printf("%dx^%d",p->coeff,p->exp);
        if(p->next)printf(" + "); p=p->next;}
}

int main(){
    struct Node *p=NULL,*q=NULL,*r;
    insert(&p,5,2);insert(&p,3,1);insert(&p,2,0);
    insert(&q,2,2);insert(&q,4,1);insert(&q,1,0);
    printf("P1 = ");display(p);
    printf("\nP2 = ");display(q);
    r=add(p,q);
    printf("\nSum = ");display(r);
}
```

Executed Output

```
$ gcc q1.c -o q1 && ./q1
P1 = 5x^2 + 3x^1 + 2x^0
P2 = 2x^2 + 4x^1 + 1x^0
Sum = 7x^2 + 7x^1 + 3x^0
```

2. Previous Node in Circular Linked List

```c
#include <stdio.h>

struct Node { int data; struct Node *next; };

int main(){
    struct Node a={10,NULL},b={20,NULL},c={30,NULL};
    a.next=&b; b.next=&c; c.next=&a;
    int x=30; struct Node *p=&a;

    do{
        if(p->next->data==x){
            printf("Previous node of %d = %d",x,p->data);
            return 0;
        }
        p=p->next;
    }while(p!=&a);

    printf("Node not found");
}
```

Executed Output



3. Insert at Given Position in Circular Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

int main(){
    struct Node a={10,NULL},b={20,NULL},c={30,NULL};
    a.next=&b;b.next=&c;c.next=&a;

    struct Node *head=&a,*p=head;
    struct Node *n=malloc(sizeof(struct Node));
    int pos=2; n->data=15;

    for(int i=1;i<pos-1;i++) p=p->next;
    n->next=p->next; p->next=n;

    p=head;
    do{printf("%d ",p->data);p=p->next;}while(p!=head);
}
```

Executed Output



4. Largest and Second Largest in a Queue

```
#include <stdio.h>

int main(){
    int q[]={10,40,20,50,30},n=5;
    int largest=-999999,second=-999999;

    for(int i=0;i<n;i++){
        if(q[i]>largest){
            second=largest; largest=q[i];
        }else if(q[i]>second && q[i]!=largest)
            second=q[i];
    }

    printf("Queue: 10 40 20 50 30\n");
    printf("Largest = %d\n",largest);
    printf("Second Largest = %d\n",second);
}
```

Executed Output



5. Insert Third Element and Compare with First

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

int main(){
    struct Node *a=malloc(sizeof(struct Node));
    struct Node *b=malloc(sizeof(struct Node));
    struct Node *c=malloc(sizeof(struct Node));

    a->data=10;b->data=20;c->data=10;
    a->next=b;b->next=NULL;

    c->next=b->next;b->next=c;

    printf("List: %d %d %d\n",a->data,b->data,c->data);

    if(a->data==c->data)
        printf("First and third elements are same");
    else
        printf("First and third elements are different");
}
```

Executed Output



6. Two Stacks in One Array

```
#include <stdio.h>

int a[10],top1=-1,top2=10;

void push1(int x){
    if(top1+1==top2) printf("Overflow\n");
    else a[++top1]=x;
}

void push2(int x){
    if(top1+1==top2) printf("Overflow\n");
    else a[--top2]=x;
}

void display(){
    printf("Stack 1: ");
    for(int i=0;i<=top1;i++)printf("%d ",a[i]);
    printf("\nStack 2: ");
    for(int i=top2;i<10;i++)printf("%d ",a[i]);
}

int main(){
    push1(10);push1(20);
    push2(50);push2(40);
    display();
}
```

Executed Output



7. Flatten a Multilevel Sorted Linked List

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next,*bottom; };

struct Node *newNode(int x){
    struct Node *n=malloc(sizeof(struct Node));
    n->data=x;n->next=n->bottom=NULL;return n;
}

struct Node *merge(struct Node *a,struct Node *b){
    if(!a)return b;if(!b)return a;
    if(a->data<b->data){
        a->bottom=merge(a->bottom,b);return a;
    }else{
        b->bottom=merge(a,b->bottom);return b;
    }
}

struct Node *flatten(struct Node *h){
    if(!h||!h->next)return h;
    h->next=flatten(h->next);
    return merge(h,h->next);
}

int main(){
    struct Node *h=newNode(5);
    h->next=newNode(10);h->next->next=newNode(19);
    h->next->next->next=newNode(28);

    h->bottom=newNode(7);h->bottom->bottom=newNode(8);
    h->next->bottom=newNode(20);
    h->next->next->bottom=newNode(22);
    h->next->next->next->bottom=newNode(35);
    h->next->next->next->bottom->bottom=newNode(40);
    h->next->next->next->bottom->bottom->bottom=newNode(45);

    h=flatten(h);
    printf("Flattened list: ");
    while(h){printf("%d ",h->data);h=h->bottom;}
}
```

Executed Output

```
$ gcc q7.c -o q7 && ./q7
Flattened list: 5 7 8 10 19 20 22 28 35 40 45
```

8. Merge K Sorted Linked Lists

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

void insert(struct Node **h,int x){
    struct Node *n=malloc(sizeof(struct Node)),*p=*h;
    n->data=x;n->next=NULL;
    if(!*h){*h=n;return;}
    while(p->next)p=p->next;
    p->next=n;
}

struct Node *merge(struct Node *a,struct Node *b){
    struct Node d,*t=&d; d.next=NULL;
    while(a&&b){
        if(a->data<b->data){t->next=a;a=a->next;}
        else{t->next=b;b=b->next;}
        t=t->next;
    }
    t->next=a?a:b;
    return d.next;
}

int main(){
    struct Node *a=NULL,*b=NULL,*c=NULL;
    insert(&a,1);insert(&a,4);insert(&a,7);
    insert(&b,2);insert(&b,5);
    insert(&c,3);insert(&c,6);

    struct Node *r=merge(merge(a,b),c);
    printf("Merged list: ");
    while(r){printf("%d ",r->data);r=r->next;}
}
```

Executed Output



9. Swap Kth Node from Beginning and End

```
#include <stdio.h>

struct Node { int data; struct Node *next; };

void swapK(struct Node *h,int k){
    int n=0; struct Node *p=h,*q=h;
    while(p){n++;p=p->next;}
    if(k>n)return;

    p=q=h;
    for(int i=1;i<k;i++)p=p->next;
    for(int i=1;i<n-k+1;i++)q=q->next;

    int t=p->data;p->data=q->data;q->data=t;
}

int main(){
    struct Node a={10,NULL},b={20,NULL},c={30,NULL};
    struct Node d={40,NULL},e={50,NULL};
    a.next=&b;b.next=&c;c.next=&d;d.next=&e;

    int k=2; swapK(&a,k);

    printf("After swapping k = %d: ",k);
    struct Node *p=&a;
    while(p){printf("%d ",p->data);p=p->next;}
}
```

Executed Output



10. Multiply Two Numbers Represented by Linked Lists

```
#include <stdio.h>
#include <stdlib.h>

struct Node { int data; struct Node *next; };

void insert(struct Node **h,int x){
    struct Node *n=malloc(sizeof(struct Node)),*p=*h;
    n->data=x;n->next=NULL;
    if(!*h){*h=n;return;}
    while(p->next)p=p->next;
    p->next=n;
}

long long value(struct Node *p){
    long long n=0;
    while(p){n=n*10+p->data;p=p->next;}
    return n;
}

int main(){
    struct Node *a=NULL,*b=NULL;

    insert(&a,1);insert(&a,2);insert(&a,3);
    insert(&b,4);insert(&b,5);

    printf("First number: 123\n");
    printf("Second number: 45\n");
    printf("Product = %lld",value(a)*value(b));
}
```

Executed Output

